Estun Codroid

远程 API 文档

目录

1. 通信基础 2

1.1 通信协议 2

1.2 使用说明 2

1.3 通用请求格式 2

1.4 通用响应格式 2

2. 工程相关接口 3

2.1 运行脚本 3

2.2 进入远程脚本模式 4

2.3 运行工程（内部接口） 4

2.4 通过工程映射索引号运行工程 5

2.5 单步运行（内部接口） 5

2.6 暂停工程 6

2.7 恢复运行工程 6

2.8 停止运行工程 6

2.9 设置断点(暂时无法使用) 7

2.10 添加断点(暂时无法使用) 7

2.11 删除断点(暂时无法使用) 8

2.12 清除所有断点(暂时无法使用) 8

2.13 设置启动行 9

2.14 清除从指定行运行 9

3. 全局变量相关接口 10

3.1 全局变量定义 10

3.2 变量名命名规则 10

3.3 获取全局变量 10

3.3 保存全局变量 11

3.4 删除全局变量 13

4. 工程变量接口 13

4.1 获取当前所有工程变量值 13

5. 末端 485 接口 14

5.1 初始化 14

5.2 清空缓存 14

5.3 读取数据 15
5.4 发送数据 15
6. ModbusTCP 主站接口 16
6.1 创建连接设备 16
6.2 删除连接设备 16
6.3 创建/修改通信表 17
6.4 删除通信表 17
6.5 修改表的通信周期 18
6.6 给地址设置别名 18
6.7 给地址段设置数据类型 19
6.8 修改地址的值 20
6.9 获取所有设备配置 20
6.10 获取所有设备状态 22
7. 负载辨识相关接口(待测试) 23
7.1 是否在给定采样位置 23
7.2 传感器数据采样 23
7.3 负载辨识计算接口 24
8. 拖动功能相关接口 25
8.1 初始化拖动参数 25
8.2 获取拖动灵敏度 26
8.3 获取拖动模式 26
8.4 开启/关闭拖动姿态锁 26
8.5 获取拖动姿态锁的状态 27
9. 传送带相关接口(待测试) 27
9.1 获取编码器计数 27
9.2 使能传送带 28
9.3 取消使能传送带 28
9.4 传送带当量标定 29
9.5 设置传送带配置参数 29
9.6 获取传送带配置参数 31
9.7 使用 TCP 连接相机 32
9.8 断开相机连接 32

9.9 获取相机状态 33
9.10 发送消息到相机 33
9.11 获取上次触发 DI 时的编码器计数 34
9.12 标定 DI 信号触发位置 34
10. 机器人计算接口 35
10.1 正解 35
10.2 逆解 35
11. 机器人运动控制接口 36
11.1 点动(暂时无法使用) 36
11.2 停止点动(暂时无法使用) 37
11.3 点动心跳(暂时无法使用) 37
11.4 RunTo 37
11.5 RunTo 心跳 38
12. 机器人控制命令 38
12.1 上使能 38
12.2 下使能 39
12.3 进入手动模式 39
12.4 进入自动模式 40
12.5 进入远程模式 40
13. IO 相关接口 41
13.1 获取 IO 值 41
13.2 写入 IO 值 41
14. 寄存器相关接口 42
14.1 获取寄存器值 42
14.2 写入寄存器值 42
15. 主题订阅/推送接口 43
15.1 主题订阅接口 43
15.2 工程状态 43
15.3 变量数据主题 44
15.4 机器人状态主题 44
15.5 机器人位姿主题 45
15.6 机器人坐标系主题 45

15.7 系统日志主题 46
15.7 系统错误主题 46
16. 远程脚本模式接口 46
16.1 进入方式 46
16.2 通信方式 47
16.3 接口说明 47
16.4 运行机制 47
17.CRI 实时控制接口(Codroid RealTime Interface) 47
17.1 实时数据定义 48
17.2 开启数据流推送 48
17.3 关闭数据流推送 48
17.3 开启实时控制 49
17.3 关闭实时控制 49

修订记录

日期	修订内容	修订人
2025.9.18	初版	陈艺堃
2025.10.20	增加传送带跟随相关接口说明	陈艺堃
2025.10.20	增加 IO、寄存器、机器人控制相关的部分接口说明	刘昌源
2025.10.27	更新 2.2.8.5、2.2.8.6	陈艺堃
2025.10.28	新增点动接口，机器人控制命令	陈艺堃
2025.11.06	格式调整	张启博
2025.11.14	修订所有接口；修订版本号变更到 v2.3 以最新软件版本命名	袁润泽

1. 通信基础

1.1 通信协议

- **协议**: TCP/IP
- **服务端**: 机器人控制器
- **端口号**: 9001（主接口），9002（远程脚本模式）
- **默认 IP**: 192.168.1.136
- **数据格式**: JSON 字符串，UTF-8 编码

1.2 使用说明

i. **连接建立**: 使用 TCP 连接到机器人控制器 IP:9001 端口

ii. **请求格式**: 所有请求必须遵循标准 JSON 格式

iii. **错误处理**: 检查响应中的 err 字段处理错误

iv. **心跳维护**: 点动和 RunTo 运动需要定期发送心跳维持

v. **数据订阅**: 使用主题订阅接口实时获取状态更新

vi. **远程脚本**: 使用 9002 端口进行远程脚本模式通信

1.3 通用请求格式

```
{
    "id": "请求 ID", // 请求 ID，用于标识请求，用户自定义，可以为数据或字符串，返回请求时原值返回
    "ty": "请求类型", // 请求类型，由相应接口定义
    "db": {} // 请求数据，具体内容由相应接口定义
}
```

1.4 通用响应格式

```
{
    "id": "请求 ID", // 请求 ID，与请求参数一致
    "ty": "请求类型", // 请求类型，与请求参数一致
    "db": {}, // 返回数据，具体内容由相应接口定义
    "err": "错误信息" // 如果请求失败，会有该字段，code 是错误码，msg 是具体的错误消息
}
```

接口没有数据返回时，默认返回数据如下，接口不再进行返回数据说明。

```
{
    "id": "请求 ID", // 请求 ID，与请求参数一致
    "ty": "请求类型", // 请求类型，与请求参数一致
```

```
}
```

2. 工程相关接口

2.1 运行脚本

接口类型: project/runScript

功能: 客户端直接发送脚本运行

请求数据:

```
{
    "id": "请求 ID",
    "ty": "project/runScript",
    "db": {
        "scripts": {
            "main": "主程序代码",
            "subThreads": {"线程名": "线程代码"},
            "subPrograms": {"程序名": "子程序代码"},
            "interrupts": {"中断名": "中断程序代码"}
        },
        "vars": {// 可选参数，脚本共用变量，多脚本共享，同工程变量
       "变量 1": "值 1", // key 为变量名，value 为变量值
       "变量 2": "值 2", // key 为变量名，value 为变量值
       }
    }
}
```

示例:

```
{
    "id": "1",
    "ty": "project/runScript",
    "db": {
        "scripts": {
            "main": {"print(v1)\nrunScript(\"t1\")\nprint(2)\n"},
            "subThreads": {"t1": "print(3)\ncallSubprogram(\"p1\")"},
            "subPrograms": {"p1": "print(v2)"},
            "interrupts": {"in1": "print(5)"}
        },
        "vars": {
        "v1": 1,
        "v2": "hello",
        }
    }
}
```

响应数据:

```
{
    "id":"1",
    "ty":"project/runScript"
}
```

2.2 进入远程脚本模式

接口类型: project/enterRemoteScriptMode

功能: 进入远程脚本模式

请求数据:

```
{
    "id": "请求 ID",
    "ty": "project/enterRemoteScriptMode"
}
```

响应数据：

```
{
    "id": "1",
    "ty": "project/enterRemoteScriptMode"
}
```

2.3 运行工程（内部接口）

接口类型: project/run

功能: 运行指定工程

注释:需要注意的是，如果脚本设置了断点，当运行到断点处时，工程自动进入暂停状态。

工程 ID 需要导出当前脚本，解压 Project.zip，projectlua 下的文件夹名即为工程 ID

注: 如果发送的工程 ID 不对，web 端会弹窗警告，但是接口不会返回错误信息

请求数据:

```
{
    "id": "请求 ID",
    "ty": "project/run",
    "db": {
        "id": "mhv9ubqz0pr69d5f" // 工程 ID
    }
}
```

响应数据：

```
{
    "id": "1",
    "ty": "project/run"
```

```
}
```

2.4 通过工程映射索引号运行工程

接口类型: project/runByIndex

功能: 通过索引号运行已绑定的工程

请求数据:

```
{
    "id": "请求 ID",
    "ty": "project/runByIndex",
    "db":  1 // 工程映射索引号，int 类型而非字符串
}
```

响应数据:

```
{
    "id": "1",
    "ty": "project/runByIndex"
}
```

2.5 单步运行（内部接口）

接口类型: project/runStep

功能: 单步运行工程代码

注释:运行一行脚本，当没有工程在运行时，需传入工程 id，当工程暂停时，再次调该接口不需要传工程 id，工程执行一行代码后再次暂停。

单步运行时，只有主任务会根据该接口，一行一行的运行（调用一次运行一行），线程和中断会正常执行，所以工程会处于运行状态。

工程 ID 需要导出当前脚本，解压 Project.zip，projectlua 下的文件夹名即为工程 ID

请求数据:

```
{
    "id": "请求 ID",
    "ty": "project/runStep",
    "db": {
        "id": "mhv9ubqz0pr69d5f" // 工程 ID
    }
}
```

响应数据:

```
{
    "id": "1",
```

```
    "ty": "project/runStep"
}
```

2.6 暂停工程

接口类型: project/pause

功能: 暂停正在运行的工程

请求数据:

```
{
    "id": "请求 ID",
    "ty": "project/pause"
}
```

响应数据:

```
{
    "id": "1",
    "ty": "project/pause",
}
```

2.7 恢复运行工程

接口类型: project/resume

功能: 恢复暂停的工程

请求数据:

```
{
    "id": "请求 ID",
    "ty": "project/resume"
}
```

响应数据:

```
{
    "id": "1",
    "ty": "project/resume",
}
```

2.8 停止运行工程

接口类型: project/stop

功能: 停止工程运行

请求数据:

```
{
    "id": "请求 ID",
    "ty": "project/stop"
}
```

响应数据：

```
{
    "id": "1",
    "ty": "project/stop",
}
```

2.9 设置断点(暂时无法使用)

接口类型: project/setBreakpoint

功能: 为脚本设置断点

注：只允许主任务设置断点。当用户切换工程或重启后，断点数据需清除。

该接口会覆盖设置对应脚本的断点。

如果要单独清除某个脚本的断点，可传递空数组。

请求数据:

```
{
    "id": "请求 ID", //请求 id
    "ty": "project/setBreakpoint",
    "db": {
        "m8wope1d0tka199a":[8,10,15],//key 为脚本 id，value 为数组，元素为行号
        "m8wope1d0tka199b":[20, 25]
     }
 }
```

响应数据：

```
{
    "id": "1",
    "ty": "project/setBreakpoint",
}
```

2.10 添加断点(暂时无法使用)

接口类型: project/addBreakpoint

功能: 添加断点(增加对应脚本的断点，即传入的与已有的合并)

请求数据:

```
{
    id: "请求 ID", //请求 id
```

```
    ty: "project/addBreakpoint",
    db: {
        m8wope1d0tka199a: [8,10, 15], // key 为脚本 id，value 为数组，元素为行号
        m8wope1d0tka199b: [20, 25]
    }
}
```

响应数据：

```
{
    "id": "1",
    "ty": "project/addBreakpoint",
}
```

2.11 删除断点(暂时无法使用)

接口类型: project/removeBreakpoint

功能: 停止工程运行

请求数据:

```
{
    id: "请求 ID", //请求 id
    ty: "project/removeBreakpoint",
    db: {
        m8wope1d0tka199a: [8, 10, 15], // key 为脚本 id，value 为数组，元素为行号
        m8wope1d0tka199b: [20, 25]
    }
}
```

响应数据：

```
{
    "id": "1",
    "ty": "project/removeBreakpoint",
}
```

2.12 清除所有断点(暂时无法使用)

接口类型: project/clearBreakpoint

功能: 停止工程运行

请求数据:

```
{
    "id": "请求 ID",
    "ty": "project/clearBreakpoint"
}
```

响应数据：

```
{
    "id": "1",
    "ty": "project/clearBreakpoint",
}
```

2.13 设置启动行

接口类型: project/setStartLine

功能: 设置主程序从哪一行开始执行

注：只有主任务可以设置从哪一行开始运行，该设置会应用到下一次工程运行。且只生效一次。

请求数据:

```
{
    "id": "请求 ID",
    "ty": "project/setStartLine",
   "db": 3 //行号
}
```

响应数据：

```
{
    "id": "1",
    "ty": "project/setStartLine",
}
```

2.14 清除从指定行运行

接口类型: project/clearStartLine

功能: 清除之前设定的从指定行运行

请求数据:

```
{
    "id": "请求 ID",
    "ty": "project/clearStartLine"
}
```

响应数据：

```
{
    "id": "1",
    "ty": "project/clearStartLine",
}
```

3. 全局变量相关接口

3.1 全局变量定义

全局变量默认是掉电保存的，且运行时的值也会保留。即全局变量的值可由用户修改或运行时被修改，并且工程结束后值不会恢复到运行前，会保留当前值。

全局变量和工程变量的值以 json 字符串的形式保存。

下面是不同类型数据的示例，所有符号都必须是英文符号，不建议使用中文：

1. 数字，直接输入数字即可：100 或 2.1
2. 字符串，必须使用双引号：" abcas"
3. 数组，使用中括号,：[2,4.1," abc"]，[[1,2,3],[4,5,6]]
4. Map：{ "jp" :[1,2,3,5,6,8]}，{ "a" :{ "b" : 1}}

3.2 变量名命名规则

变量名需符合 lua 语法规范，及以字母或下划线开头。

避免使用双下划线开头，避免使用系统保留关键字及函数名。

主要保留字如下：

```
"and", "break", "do", "else", "elseif", "end","false", "for", "function", "goto", "if", "in","local", "nil", "not", "or", "repeat",
"return","then", "true", "until", "while", "table", "math","DO", "DOGroup", "DIO", "DIOGroup", "AO",
"AIO","ModbusTCP","setSpeedJ", "setAccJ", "setSpeedL", "setAccL", "setBlender","setMoveRate","getCoor",
"getTool", "setCoor", "editCoor", "setTool", "editTool","setPayload","enableVibrationSuppression",
"disableVibrationSuppression","setCollisionDetectionSensitivity","initComplianceControl",
"enableComplianceControl","disableComplianceControl","forceControlZeroCalibrate",
"setFilterPeriod","searchSuccessed","getJoint", "getTCP", "getCoor", "getTool", "aposToCpos","cposToApos",
"cposToCpos","posOffset", "posTrans", "coorRel", "toolRel", "getJointTorque","getJointExternalTorque","createTray",
"getTrayPos", "posInverse", "distance", "interPos","planeTrans","getTrajStart", "getTrajEnd", "arrayAdd",
"arraySub","coorTrans","movJ", "movL", "movC", "movCircle", "movLW", "movCW", "movTraj","setWeave",
"weaveStart", "weaveEnd","setDO", "getDI", "getDO", "setDOGroup", "getDIGroup","getDOGroup", "setAO", "getAI",
"getAO","getRegisterBool", "setRegisterBool", "getRegisterInt","setRegisterInt", "getRegisterFloat",
"setRegisterFloat","RS485init", "RS485flush", "RS485write", "RS485read","readCoils", "readDiscreteInputs",
"readHoldingRegisters","readInputRegisters","writeSingleCoil", "writeSingleRegister",
"writeMultipleCoils","writeMultipleRegisters","createSocketClient", "connectSocketClient",
"writeSocketClient","readSocketClient", "closeSocketClient","wait", "waitCondition", "systemTime",
"stopProject","pauseProject", "runScript", "pauseScript", "resumeScript","stopScript", "callModule",
"print","setInterruptInterval", "setInterruptCondition","clearInterrupt","strcmp", "strToNumberArray",
"arrayToStr","enableMultiWeld", "getCurSeam", "isMultiWeldFinished","setMultiWeldOffset", "weldNextSeam",
"resetMultiWeld","searchStart", "setMasterFlag", "getOffsetValue", "search","searchEnd", "searchOffset",
"searchOffsetEnd", "searchError"
```

3.3 获取全局变量

接口类型: globalVar/getVars

功能: 获取所有全局变量

请求数据:

```
{
    "id": "请求 ID",
    "ty": "globalVar/getVars"
}
```

响应数据:

```
{
    "id": "1",
    "ty": "globalVar/getVars",
    "db": {
        "Test_str": {
            "val": "\"100000\"",
            "nm": "这是一个字符串"
        },
        "ps1": {
            "val": "{\"a\":1}",
            "nm": ""
        },
        "qqq": {
            "val": "1",
            "nm": ""
        },
        "v991": {
            "val": "100",
            "nm": " "
        },
        "v992": {
            "val": "90.4",
            "nm": " "
        },
        "v993": {
            "val": "[1,2,3,4,5]",
            "nm": " "
        },
        "v994": {
            "val": "{\"aaa\":100}",
            "nm": " "
        }
    }
}
```

3.3 保存全局变量

接口类型: globalVar/saveVars

功能: 保存全局变量（增量保存）

注释： 如果试图添加已存在的变量，该接口会更新同名变量的值

请求数据:

```
{
    "id": "请求 ID",
    "ty": "globalVar/saveVars",
    "db": {
        "变量名": {
            "nm": "变量备注", //不传则不处理
            "val": "变量值"
        }
    }
}
```

示例:

```
{
    "id": "1",
    "ty": "globalVar/saveVars",
    "db": {
        "Test_str": {
            "val": "\"100000\"",
            "nm": "这是一个字符串"
        },
       "v991": {
           "val": "100",
           "nm": "这是一个整数"
       },
        "v992": {
            "val": "90.4",
            "nm": "这是一个浮点数"
        },
        "v993": {
            "val": "[1, 2, 3, 4, 5]",
            "nm": "这是一个列表"
        },
        "v994": {
            "val": "{\"aaa\": 100}",
            "nm": "这是一个键值对"
        },
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "globalVar/saveVars"
```

```
}
```

3.4 删除全局变量

接口类型: globalVar/removeVars

功能: 删除指定全局变量

注释: 如果试图删除不存在的变量，该接口并不会报错

请求数据:

```
{
    "id": "请求 ID",
    "ty": "globalVar/removeVars",
    "db": ["v991", "v992"] // 变量名列表
}
```

响应数据:

```
{
    "id": "1",
    "ty": "globalVar/removeVars"
}
```

4. 工程变量接口

4.1 获取当前所有工程变量值

接口类型: globalVar/GetProjectVarUpdate

功能: 获取工程运行时变量

注释: 该接口必须在工程状态为运行时才有效，该接口不能获取全局变量，不能获取工程变量的注释

请求数据:

```
{
    "id": "请求 ID",
    "ty": "globalVar/GetProjectVarUpdate"
}
```

响应数据:

```
{
    "id": "1",
    "ty": "globalVar/GetProjectVarUpdate",
    "db": {
        "project/point/p1": { //点位信息
            "coord": 0,
            "cp": [216.2,-485.345,436.12,-175.22,1.657,-68.624], //笛卡尔坐标
```

```
                "ep": [],
                "rj": [-88.342,0.882,89.009,-3.283,86.059,-109.757],
                "tool": 0
            },
            "project/point/p2": { //点位信息
                "coord": 0,
                "ep": [],
                "jp": [-88.342,14.56,121.328,42.713,86.059,-109.757], //关节角度
                "tool": 0
            },
          "project/var/var1":"0",
    }
}
```

5. 末端 485 接口

5.1 初始化

接口类型: EC2RS485/init

功能: 初始化末端 485 接口

请求数据:

```
{
    "id": "请求 ID",
    "ty": "EC2RS485/init",
    "db": {
        "baudrate": 115200, //波特率:230400,128000,115200,57600,56000,38400,19200,14400,9600,4800,2400,1200,600,300,110
        "stopBit": 1, //可选参数，停止位：1 或 2，默认 1
        "dataBit": 8, //可选参数，数据位：固定为 8 位
        "parity": 0 //可选参数，校验位：0-无校验，1-奇校验，2-偶校验，默认 0
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "EC2RS485/init"
}
```

5.2 清空缓存

接口类型: EC2RS485/flushReadBuffer

功能: 清空读取缓存

请求数据:

```
{
    "id": "请求 ID",
    "ty": "EC2RS485/flushReadBuffer"
}
```

响应数据:

```
{
    "id": "1",
    "ty": "EC2RS485/flushReadBuffer"
}
```

5.3 读取数据

接口类型: EC2RS485/read

功能: 读取 485 数据

请求数据:

```
{
    "id": "请求 ID",
    "ty": "EC2RS485/read",
    "db": {
        "length": 8, //读取字节数，最大 128 字节
        "timeout": 3000 //可选参数，读取超时时间，单位为毫秒，范围 0~3000，默认 3000
    }
}
```

响应数据:

```
{
   "id":"m912rb1b0wsc2742",
   "type": None,
   "db":[2,10] //读取失败返回空数组，读取成功返回读取到的数据，数据为每个字节的无符号整数值
}
```

5.4 发送数据

接口类型: EC2RS485/write

功能: 通过 485 发送数据

请求数据:

```
{
    "id": "请求 ID",
    "ty": "EC2RS485/write",
    "db": [2,10]   //要发送的数据，数据为每个字节的无符号整数值,最大长度为 127
```

```
}
```

响应数据:

```
{
    "id": "1",
    "ty": "EC2RS485/write"
}
```

6. ModbusTCP 主站接口

6.1 创建连接设备

接口类型: ModbusTcp/setDevice

功能: 创建 ModbusTCP 设备连接

注释: 当试图创建同名设备时，会覆盖原先设备设置

请求数据:

```
{
    "id": "请求 ID",
    "ty": "ModbusTcp/setDevice",
    "db": {
        "name": "deviceName",// 设备名称,唯一值
        "ip": "192.168.1.100",// 设备 IP 地址
        "port": 502,// 设备端口号
        "slaveId": 1,// 从机地址
        "endian": 1// 字节序：1-大端，2-小端，默认 1
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "ModbusTcp/setDevice"
}
```

6.2 删除连接设备

接口类型: ModbusTcp/removeDevice

功能: 删除 ModbusTCP 设备

注释: 当试图删除不存在设备时，该指令不会报错

请求数据:

```
{
```

```
    "id": "请求 ID",
    "ty": "ModbusTcp/removeDevice",
    "db": {
        "name": "deviceName" //设备名称
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "ModbusTcp/removeDevice"
}
```

6.3 创建/修改通信表

接口类型: ModbusTcp/setTable

功能: 创建或修改通信表

注释: 如果试图在不存在的设备中创建表，该接口不会报错

请求数据:

```
{
    "id": "请求 ID",
    "ty": "ModbusTcp/setTable",
    "db": {
        "name": "设备名", // 设备名
        "tableName": "表名", // 表名，该设备下唯一
        "functionCode": 1, , // 功能码，支持 0x01，0x02，0x03，0x04，0x05， 0x06，0x0F，0x10
        "addr": 0, // 起始地址
        "count": 10, // 读写地址数量
        "period": 1000 // 通信周期，单位为毫秒，实际周期受通信链路负载影响
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "ModbusTcp/setTable"
}
```

6.4 删除通信表

接口类型: ModbusTcp/removeTable

功能: 删除通信表

注释: 如果试图在不存在的设备中删除表，或者删除不存在的表，该接口不会报错

请求数据:

```
{
    "id": "请求 ID",
    "ty": "ModbusTcp/removeTable",
    "db": {
        "name": "deviceName", //设备名
       "tableName":"tableName"//表名
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "ModbusTcp/removeTable"
}
```

6.5 修改表的通信周期

接口类型: ModbusTcp/setPeriod

功能: 修改表的通信周期

注释: 如果试图修改不存在的表，该接口不会报错

请求数据:

```
{
    "id": "请求 ID",
    "ty": "ModbusTcp/setPeriod",
    "db": {
        "name": "deviceName", //设备名
        "tableName":"tableName", //表名
       "period":800//正整数，周期，单位为毫秒
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "ModbusTcp/setPeriod"
}
```

6.6 给地址设置别名

接口类型: ModbusTcp/setName

功能: 给地址设置别名

注释：如果试图修改不存在的表，该接口不会报错

请求数据:

```
{
    "id": "请求 ID",
    "ty": "ModbusTcp/setName",
    "db": {
        "name": "设备名", // 设备名
        "tableName": "表名", // 表名，该设备下唯一
        "addr": 0, // 地址
        "alias": "flag" // 别名
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "ModbusTcp/setName"
}
```

6.7 给地址段设置数据类型

接口类型: ModbusTcp/setType

功能: 给地址段设置数据类型

注释：如果试图修改不存在的表，该接口不会报错

只对保持和输入寄存器有效，对应功能码 0x03,0x04,0x06,0x10。

默认数据类型是 U16。

注意：0x06 功能码只有一个地址，所以只能设置为 I16 或 U16。

请求数据:

```
{
    "id": "请求 ID",
    "ty": "ModbusTcp/setType",
    "db": {
        "name": "设备名", // 设备名
        "tableName": "表名", // 表名，该设备下唯一
        "type": "I32",//数据类型：I16,I32,I64,U16,U32,U64,F32,F64
        "addr": 0, // 起始地址，一个地址对应两个字节长度，即 16 位
        "count": 10 // 地址数量，该数量应与对应的数据类型长度一致，可连续设置多组，例如将 4 个地址设置为两个 U32 类型数据
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "ModbusTcp/setType"
}
```

6.8 修改地址的值

接口类型: ModbusTcp/setVal

功能: 修改地址的值

注释: 只有写表的数据值可修改

请求数据:

```
{
    "id": "请求 ID",
    "ty": "ModbusTcp/setVal",
    "db": {
        "name": "设备名", // 设备名
        "tableName": "表名", // 表名，该设备下唯一
        "addr": 0, // 地址，如果设置过数据类型，该地址须是该类型数据的首地址
       "val":33 //该数据类型对应的值，线圈寄存器的值为 0 或
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "ModbusTcp/setVal"
}
```

6.9 获取所有设备配置

接口类型: ModbusTcp/getConfig

功能: 获取所有设备配置

请求数据:

```
{
    "id": "请求 ID",
    "ty": "ModbusTcp/getConfig",
}
```

响应数据:

```
{
   "id":"1",
    "ty":"ModbusTcp/getConfig",
```

```
    "db":{
        "deviceefx":{//设备名
            "ip": "192.168.1.150",// IP
            "port":502,//端口
            "endian":1,//字节序：1-大端，0-小端
            "slaveId": 1,//从机地址
            "tables":{ //表配置
                "table37b":{//表名
                    "functionCode": 16, //功能码
                    "addr":0,//起始地址
                    "count":10, //读写数量
                    "period": 1000, //通信周期，单位为毫秒
                    "alias":{//地址别名
                        "flag": 0// key 为别名，value 为地址
                    },
                    "val": [//地址值，功能码为 0x05,0x0F 时，值为 0 或 1，功能码为 0x06,0x10 时,值为 U16 类型，其他功能码无该字段
                        0,
                        8448,
                        0,
                        8448,
                        0,
                        0,
                        19713,
                        0,
                        0,
                        0
                    ],
                    "dataFormat":{//数据类型,功能码为 0x03,0x04,0x06,0x10 时有效
                        "0":3,
                        "1":0, // 0 无类型（与前面的子地址组合为其他类型）
                        "2":2, // 1:I16,2:U16,3:I32,4:U32,5:I64,6:U64,7:F32,8:f64
                        "3":2,
                        "4":2,
                        "5":2,
                        "6":2,
                        "7":2,
                        "8":2,
                        "9":2
                    }
                },
                "tablez1q":{//表名
                    "functionCode": 1, //功能码
                    "addr":0,//起始地址
                    "count":10,//读写数量
                    "period": 800,//通信周期，单位为毫秒
                    "alias":{ //地址别名
                        "Name1": 15 // key 为别名，value 为地址
                    },
                    "val": [],
```

```
                "dataFormat":{}
            }
        }
    }
}
```

6.10 获取所有设备状态

接口类型: ModbusTcp/getState

功能: 获取所有设备状态

请求数据:

```
{
    "id": "请求 ID",
    "ty": "ModbusTcp/getState"
    }
}
```

响应数据:

```
{
    "id":"1",
    "ty":"ModbusTcp/getState",
    "db":{
        "deviceefx":{//设备名
            "state":1, //连接状态，0：未连接 1：已连接
            "tables":{ //表数据
                "table37b":{//表名
                    "syncCount": 1923,//同步次数
                    "syncError": 0, //同步错误次数
                    "val": [//地址值
                        33, //该地址对应数据类型的值
                        "--",//该地址与上面的地址合并为新的数据类型
                        0,
                        33,
                        0,
                        0,
                        333,
                        0,
                        0,
                        0
                    ]
                },
                "tablez1q":{//表名
                    "syncCount": 3064,//同步次数
                    "syncError": 1,//同步错误次数
                    "val": [//地址值
```

```
                    0,
                    1,
                    0,
                    0,
                    0,
                    0,
                    0,
                    0,
                    0,
                    0
                ]
            }
        }
    }
}
```

7. 负载辨识相关接口(待测试)

7.1 是否在给定采样位置

接口类型: Robot/IsOnGivenSamplePositon

功能: 检查机器人是否在指定采样位置

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/IsOnGivenSamplePositon",
    "db": {
        "jntTargetPos": [0, 90, 180, 0, 0, 0], //Float64,单位:degree,数组长度为机器人的自由度
        "jntActualPos": [0, 90.05, 179.99, 0, 0, 0] // Float64,单位:degree,数组长度为机器人的自由度
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Robot/IsOnGivenSamplePositon",
    "db":true //true:机器人在给定的采样位置,false:机器人不在给定的采样位置
}
```

7.2 传感器数据采样

接口类型: Robot/LoadIdenJSSample

功能: 采集负载辨识传感器数据

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/LoadIdenJSSample"
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Robot/LoadIdenJSSample",
    "db": {
        "data": [0.1, 40.1, 23.5, 10.2, 0.9, 0.3],//Float64,单位:Nm,数组长度为机器人的自由度
        "isSampleAcceptable": true//false:接近奇异位,采样数据不可用; true:采样数据可用
    }
}
```

7.3 负载辨识计算接口

接口类型: Robot/PayloadIdentificationJS

功能: 负载辨识计算

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/PayloadIdentificationJS",
   "db": [ // 4 个采样点的数据
      {
       "jntPos":[0,0,90,0,90,0], //Float64,单位:degree,数组长度为机器人的自由度
       "jntTrqWithoutLoad":[1,2,3,4,5,6],//Float64,单位:Nm,数组长度为机器人的自由度
       "jntTrqWithLoad":[1,2,3,4,5,6] //Float64,单位:Nm,数组长度为机器人的自由度
       },
       {
       "jntPos":[0,0,90,0,90,0],
       "jntTrqWithoutLoad":[1,2,3,4,5,6],
       "jntTrqWithLoad":[1,2,3,4,5,6]
       },
       {
       "jntPos":[0,0,90,0,90,0],
       "jntTrqWithoutLoad":[1,2,3,4,5,6],
       "jntTrqWithLoad":[1,2,3,4,5,6]
       },
       {
       "jntPos":[0,0,90,0,90,0],
       "jntTrqWithoutLoad":[1,2,3,4,5,6],
       "jntTrqWithLoad":[1,2,3,4,5,6]
       }
   ]
```

```
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Robot/PayloadIdentificationJS",
    "db": {
        "succeed": true, //辨识是否成功
        "isResultsAcceptable": true, //辨识结果是否可信
        "m": 10.32254447, //负载质量，单位:kg
        "pos": { //负载质心位置，单位:mm
            "mx": 10.36554,
            "my": 10,
            "mz": 10
        },
        "maxError": { //最大误差
            "Force": 10.55874444, // 单位:N
            "Torque": 10 // 单位:Nm
        },
        "aveError": { //平均误差
            "Force": 10.333554487, // 单位:N
            "Torque": 10 // 单位:Nm
        }
    }
}
```

8. 拖动功能相关接口

8.1 初始化拖动参数

接口类型: Robot/SetDefaultServoDragParam

功能: 初始化拖动参数

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/SetDefaultServoDragParam"
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Robot/SetDefaultServoDragParam",
    "db": {
        "state": true, //是否设置成功
        "text": "拖动参数初始化成功." //向前端推送需要显示的信息
```

```
    }
}
```

8.2 获取拖动灵敏度

接口类型: Robot/GetDragSensitivity

功能: 获取当前拖动灵敏度

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/GetDragSensitivity"
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Robot/GetDragSensitivity",
    "db": {
        "sensitivity": 100 //拖动灵敏度 0-100
    }
}
```

8.3 获取拖动模式

接口类型: Robot/GetDragMode

功能: 获取当前拖动模式

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/GetDragMode"
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Robot/GetDragSensitivity",
    "db": 0 //拖动模式 0->老版拖动, 2->新版拖动
}
```

8.4 开启/关闭拖动姿态锁

接口类型: Robot/SetCartOriLock

功能: 开启/关闭拖动姿态锁

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/SetCartOriLock",
   "db":{
       "cartOriLock": true    // true 开启, false 关闭
   }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Robot/SetCartOriLock",
     "db": true //是否开启成功
}
```

8.5 获取拖动姿态锁的状态

接口类型: Robot/GetCartOriLockState

功能: 获取拖动姿态锁的状态

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/GetCartOriLockState"
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Robot/GetCartOriLockState",
    "db": true    //是否开启成功
}
```

9. 传送带相关接口(待测试)

9.1 获取编码器计数

接口类型: Conveyor/getEncoderCount

功能: 获取传送带编码器当前计数值

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Conveyor/getEncoderCount",
    "db": {
        "index": 1 // 传送带编号
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Conveyor/getEncoderCount",
    "db": 100 // 传送带当前编码器计数
}
```

9.2 使能传送带

接口类型: Conveyor/enableConveyor

功能: 使能后，控制器通过高速 IO 来同步传送带编码器位置

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Conveyor/enableConveyor",
    "db": {
        "index": 1 // 传送带编号
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Conveyor/enableConveyor"
}
```

9.3 取消使能传送带

接口类型: Conveyor/disableConveyor

功能: 取消使能传送带

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Conveyor/disableConveyor",
    "db": {
```

```
        "index": 1 // 传送带编号
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Conveyor/disableConveyor"
}
```

9.4 传送带当量标定

接口类型: Conveyor/calibrateConveyor

功能: 传送带当量标定

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Conveyor/calibrateConveyor",
    "db": {
    "index": 1, // 传送带编号
    "start" : {
        "point": [1,2,3,4,5,6], // 起点处机器人示教的笛卡尔坐标
        "count": 100 // 起点处编码器计数
    },
    "end" : {
        "point": [1,2,3,4,5,6], // 终点处机器人示教的笛卡尔坐标
        "count": 100 // 终点处编码器计数
    }
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Conveyor/calibrateConveyor",
    "db": 11.2 // 传送带当量标定结果
}
```

9.5 设置传送带配置参数

接口类型: Conveyor/setConfig

功能: 设置传送带配置参数

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Conveyor/setConfig",
   "db": {
       "index": 1, // 传送带编号
       "type": 1, // 传送带类型 1：直线传送带 2：圆盘传送带
       "triggerType": 0, // 触发方式 0：无 1：DI 触发 2: 相机触发
       "conveyorCoorId": 1, // 传送带坐标系编号
       "encoderResolution": 0.00002, // 编码器分辨率
       "encoderChannel": 1, // 编码器通道号 1: HDI1&HDI2 2: HDI3&HDI4
       "syncDeviation": 10.002, // 跟随允差，单位 mm
       "minBoundary": 10.01, // 最小边界，单位 mm
       "latestSyncStart": 40.04, // 最晚启动点，单位 mm
       "latestSyncEnd": 80.08, // 最晚同步点，单位 mm
       "maxBoundary": 110.02, // 最大边界，单位 mm
       "DI": {
           "port": 0, // DI 端口号
           "triggerPos": 100.2, // 设置 DI 触发位置
       },
       "camera": {
           "ip": "192.168.1.150", // 相机 IP 地址
           "port": 8080, // 相机端口号
           "type": 0, // 相机类型 0：2D 1：3D
           "duplicateDis": 100, // 相机去重距离，单位 mm
           "isDOTrigger": true, // 是否使用 DO 触发相机拍照
           "DOPort": 1, // DO 端口号
           "retentTime": 100, // DO 高电平持续时间，单位 ms
           "interval": 10000, // 相机拍照触发间隔，单位 ms
       },
       "sheild": {
           "time": 1000, // 屏蔽时间，单位 ms
           "distance": 100.2, // 屏蔽距离，单位 mm
       },
       "offset": {
           "x": 10.1, // x 方向偏移，单位 mm
           "y": 20.2, // y 方向偏移，单位 mm
           "z": 30.3, // z 方向偏移，单位 mm
           "a": 20.3, // 绕 x 轴旋转补偿角度，单位°
           "b": 30.4, // 绕 y 轴旋转补偿角度，单位°
           "c": 40.5, // 绕 z 轴旋转补偿角度，单位°
       }
   }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Conveyor/setConfig"
}
```

9.6 获取传送带配置参数

接口类型: Conveyor/calibrateConveyor

功能: 获取传送带配置参数

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Conveyor/getConfig",
    "db": {
    "index": 1 // 传送带编号
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Conveyor/getConfig",
    "db": {
        "type": 1, // 传送带类型 1：直线传送带 2：圆盘传送带
        "triggerType": 0, // 触发方式 0：无 1：DI 触发 2: 相机触发
        "conveyorCoorId": 1, // 传送带坐标系编号
        "conveyorEnable": true, // 传送带是否启用
        "encoderResolution": 0.00002, // 编码器分辨率
        "encoderChannel": 1, // 编码器通道号 1: HDI1&HDI2 2: HDI3&HDI4
        "syncDeviation": 10.002, // 跟随允差，单位 mm
        "minBoundary": 10.01, // 最小边界，单位 mm
        "latestSyncStart": 40.04, // 最晚启动点，单位 mm
        "latestSyncEnd": 80.08, // 最晚同步点，单位 mm
        "maxBoundary": 110.02, // 最大边界，单位 mm
    "DI": {
        "port": 0, // DI 端口号
        "triggerPos": 100.2, // 设置 DI 触发位置
    },
    "camera": {
        "enable": true, // 是否启用相机
        "ip": "192.168.1.150", // 相机 IP 地址
        "port": 8080, // 相机端口号
        "type": 0, // 相机类型 0：2D 1：3D
        "duplicateDis": 100, // 相机去重距离，单位 mm
        "isDOTrigger": true, // 是否使用 DO 触发相机拍照
        "DOPort": 1, // DO 端口号
        "retentTime": 100, // DO 高电平持续时间，单位 ms
        "interval": 10000, // 相机拍照触发间隔，单位 ms
    },
    "sheild": {
        "time": 1000, // 屏蔽时间，单位 ms
        "distance": 100.2, // 屏蔽距离，单位 mm
    },
```

```
    "offset": {
        "x": 10.1, // x 方向偏移，单位 mm
        "y": 20.2, // y 方向偏移，单位 mm
        "z": 30.3, // z 方向偏移，单位 mm
        "a": 20.3, // 绕 x 轴旋转补偿角度，单位°
        "b": 30.4, // 绕 y 轴旋转补偿角度，单位°
        "c": 40.5, // 绕 z 轴旋转补偿角度，单位°
    }
    }
}
```

9.7 使用 TCP 连接相机

接口类型: Conveyor/connectCamera

功能: 使用 TCP 连接相机

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Conveyor/connectCamera",
    "db": {
    "index": 1 // 传送带编号
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Conveyor/connectCamera"
}
```

9.8 断开相机连接

接口类型: Conveyor/disconnectCamera

功能: 传送带当量标定

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Conveyor/disconnectCamera",
    "db": {
    "index": 1 // 传送带编号
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Conveyor/disconnectCamera"
}
```

9.9 获取相机状态

接口类型: Conveyor/getCameraState

功能: 传送带当量标定

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Conveyor/getCameraState",
    "db": {
    "index": 1 // 传送带编号
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Conveyor/getCameraState",
    "db": {
      "state": 1, // 相机状态 0：未连接 1：已连接
      "msgHistory": [ // 收到的相机历史消息
          "...", // 消息内容
          "..."
      ]
   }
}
```

9.10 发送消息到相机

接口类型: Conveyor/sendMsgToCamera

功能: 传送带当量标定

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Conveyor/sendMsgToCamera",
    "db": {
    "index": 1, // 传送带编号
    "msg": "xxx" // 要发送的消息
```

```
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Conveyor/sendMsgToCamera"
}
```

9.11 获取上次触发 DI 时的编码器计数

接口类型: Conveyor/getDItriggerEncoderCount

功能: 传送带当量标定

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Conveyor/getDItriggerEncoderCount",
    "db": {
    "index": 1 // 传送带编号
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Conveyor/getDItriggerEncoderCount",
    "db": 100 // 上一次触发 DI 时的编码器计数
}
```

9.12 标定 DI 信号触发位置

接口类型: Conveyor/calibrateDItriggerPos

功能: 传送带当量标定

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Conveyor/calibrateDItriggerPos",
    "db": {
    "index": 1, // 传送带编号
   "x": 100.2, // 示教工件的 x 坐标
   "startCount": 100, // DI 信号触发时的编码器计数
   "endCount": 200, // 示教工件时的编码器计数
    }
```

```
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Conveyor/calibrateDItriggerPos",
    "db": 22.3 // DI 信号触发时的 x 坐标
}
```

10. 机器人计算接口

10.1 正解

接口类型: Robot/apostocpos

功能: 关节空间到笛卡尔空间正解计算

注意

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/apostocpos",
    "db": {
        "jp": [10,20,30,40,50,60], // 关节角,单位: deg
        "coor": [100,200,300,10,20,30], // 用户坐标系
        "tool": [100,200,300,10,20,30], // 工具坐标系
        "ep": []
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Robot/apostocpos",
    "db": [100,200,300,10,20,30] // 笛卡尔坐标
}
```

10.2 逆解

接口类型: Robot/cpostoapos

功能: 笛卡尔空间到关节空间逆解计算

注意：参考关节角默认为[20，20，20，20，20，20]，如果返回值为空，尝试修改 rj 参数

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/cpostoapos",
    "db": {
        "cp": [100,200,300,40,50,60],  //末端位置,单位: mm, deg
        "rj": [10,20,30,40,50,60], //参考关节角,单位: deg
        "ep": [] //可选,附加轴位置
    }
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Robot/cpostoapos",
    "db": [10,20,30,10,20,30] //逆解后的关节角,单位: deg
}
```

11. 机器人运动控制接口

11.1 点动

接口类型: `Robot/jog`

功能: 启动机器人点动

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/jog",
    "db": {
        "mode": 2, //1：关节点动 2：直线点动
        "speed": -0.1, //速度，取值范围-1~1
        "index": 3,// 关节序号，如果是关节角度 1~6 代表轴 1~轴 6，如果是直线点动，则依次为 xyzabc
        "coorType": 0,
        "coorId": 1 //用户坐标系 id
    }
}
```

注意: 需要每 500ms 调用点动心跳接口维持点动

响应数据:

```
{
    "id": "1",
    "ty": "Robot/jog",
    "db": null
}
```

11.2 停止点动

接口类型: Robot/stopJog

功能: 停止点动

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/stopJog",
    "db": ""
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Robot/stopJog",
    "db": null
}
```

11.3 点动心跳

接口类型: Robot/jogHeartbeat

功能: 维持点动状态，需在使用点动功能后每隔 0.5s 发送一次

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/jogHeartbeat",
    "db": ""
}
```

响应数据:

```
{
    "id": "1",
    "ty": "Robot/jog",
    "db": null
}
```

11.4 RunTo

接口类型: Robot/moveTo

功能: 运动到指定位置

注意: 启动 RunTo 后需要每隔 0.5S 发送一次心跳接口维持点动

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/moveTo",
    "db": {
        "type": 0,
        "target": {
            "cp": [x,y,z,a,b,c],
            "jp": [0,0.1,0.2,0.3,0.4,0.5],
            "ep": []
        }
    }
}
```

请求数据:

```
{
    "id": "1",
    "ty": "Robot/moveTo",
    "db": null
}
```

11.5 RunTo 心跳

接口类型: Robot/moveToHeartbeat

功能: 维持 RunTo 运动，启动 RunTo 后需要每隔 0.5S 发送一次心跳接口维持点动

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/moveToHeartbeat"
}
```

请求数据:

```
{
    "id": "1",
    "ty": "Robot/moveToHeartbeat",
    "db": null
}
```

12. 机器人控制命令

12.1 上使能

接口类型: Robot/switchOn

功能: 机器人上使能

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/switchOn",
    "db": ""
}
```

响应数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/switchOn",
    "db": null
}
```

12.2 下使能

接口类型: Robot/switchOff

功能: 机器人下使能

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/switchOff",
    "db": ""
}
```

响应数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/switchOff",
    "db": null
}
```

12.3 进入手动模式

接口类型: Robot/toManual

功能: 切换到手动模式

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/toManual",
    "db": ""
```

```
}
```

响应数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/toManual",
    "db": null
}
```

12.4 进入自动模式

接口类型: Robot/toAuto

功能: 切换到自动模式

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/toAuto",
    "db": ""
}
```

响应数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/toAuto",
    "db": null
}
```

12.5 进入远程模式

接口类型: Robot/toRemote

功能: 切换到远程模式

请求数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/toRemote",
    "db": ""
}
```

响应数据:

```
{
    "id": "请求 ID",
    "ty": "Robot/toRemote",
```

```
    "db": null
}
```

13. IO 相关接口

13.1 获取 IO 值

接口类型: IOManager/GetIOValue

功能: 获取多个 IO 的当前值

请求数据:

```
{
    "id": "请求 ID",
    "ty": "IOManager/GetIOValue",
    "db": [ // 传入要查询的 IO 的类型和端口号
        {"type": "DI", "port": 0},
        {"type": "DO", "port": 10},
        {"type": "AI", "port": 1},
        {"type": "AO", "port": 2}
    ]
}
```

响应数据:

```
{
    "id": "请求 ID",
    "ty": "IOManager/GetIOValue",
    "db": [
        {"type": "DI", "port": 0, "value": 0},
        {"type": "DO", "port": 10, "value": 1},
        {"type": "AI", "port": 1, "value": 2.2},
        {"type": "AO", "port": 2, "value": 4.44}
    ]
}
```

13.2 写入 IO 值

接口类型: IOManager/SetIOValue

功能: 设置 IO 输出值

请求数据:

```
{
    "id": "请求 ID",
    "ty": "IOManager/SetIOValue",
    "db": {"type": "DO", "port": 10, "value": 1}
```

```
}
```

响应数据:

```
{
    "id": "请求 ID",
    "ty": "IOManager/GetIOValue"
}
```

14. 寄存器相关接口

14.1 获取寄存器值

接口类型: IOManager/GetRegisterValue

功能: 获取寄存器值

请求数据:

```
{
    "id": "请求 ID",
    "ty": "IOManager/GetRegisterValue",
    "db": [10000, 20000] // 传入地址
}
```

响应数据:

```
{
    "id": "请求 ID",
    "ty": "RegisterManager/GetRegisterValue",
    "db": [
        {"address": 10000, "value": 0},
        {"address": 20000, "value": 0}
    ]
}
```

14.2 写入寄存器值

接口类型: RegisterManager/SetRegisterValue

功能: 设置寄存器值

请求数据:

```
{
    "id": "请求 ID",
    "ty": "RegisterManager/SetRegisterValue",
    "db": {"address": 10000, "value": 0}
}
```

请求数据:

```
{
    "id": "请求 ID",
    "ty": "RegisterManager/SetRegisterValue"
}
```

15. 主题订阅/推送接口

15.1 主题订阅接口

接口类型: publish/topic

功能: 订阅数据推送主题

注:

1. 订阅的数据只有在数据发生变化或开始订阅时推送。

2. 受数据本身变化周期影响，正常情况下数据实际推送周期会大于订阅周期。

请求数据:

```
{
    "ty": "publish/topic", // 其中 topic 为实际要订阅的主题名称
    "tc": 0 // 可选参数， 订阅周期，单位为毫秒，默认 0，实际推送周期受主题本身数据变化周期影响
}
```

响应数据:

```
{
    "ty": "publish/topic", // 与订阅名称一致
    "db": {} // 推送数据，具体内容由主题决定
}
```

15.2 工程状态

主题名称: publish/ProjectState

推送数据:

```
{
    ty: "publish/ProjectState",
    db: {
        id: "1", // id
        state: 3, // 工程状态，0：空闲，1：正在加载工程，2：正在运行，3：暂停
        isStep: false, // 是否单步运行
        projectType: 0, // 工程类型，0：普通工程 1：远程执行脚本 2：远程进入远程脚本模式
        scripts: { // 脚本数据，state 为 2 或 3 时，才有该字段
            m8wope1d0tka199a: { // 脚本 id
                line: 8 // 脚本当前正在执行的行号
```

```
            }
        }
    }
}
```

15.3 变量数据主题

主题名称: publish/VarUpdate

注意：当工程运行时，全局变量和工程变量变化时推送。只推送变化的变量。

推送数据:

```
{
    "ty": "publish/VarUpdate",
    "db": {
        "global/var/bool1": "34",// key 为 {定义域}/{类型}/{变量名}
        "project/var/dvs": "3"
    }
}
```

推送数据如上，下面对 key 的 3 个部分进行说明：

1. 定义域：global 表示是全局变量，project 表示是工程变量
2. 类型：var 表示是普通变量，point 表示是点变量
3. 变量名：变量的 key 值，不是“nm “

15.4 机器人状态主题

主题名称: publish/RobotStatus

注意：程序处于自动运行状态，state=2

推送数据:

```
{
    "ty": "publish/RobotStatus",
    "db": {
        "mode": 0, // 0=手动; 1=自动; 2=远程
        "state": 0, // 0=未使能; 1=使能中; 2=空闲; 3=点动中; 4=RunTo; 5=拖动中
        "moveRate": 1, // 自动运行速度倍率
        "manualMoveRate": 0.3, // 手动运行速度倍率
        "recoveryState": 0,
        "isSimulation": true, // 是否仿真模式
        "teachingPendant": 0, // 是否使用示教器
        "rescueFlag": 0,
        "modeSwitch": 0,
        "ToolId": 0, // 当前工具 ID
        "PayloadId": 0, // 当前负载 ID
        "CoordinateId": 1, // 当前坐标系 ID
```

```
        "defaultToolId": 0, // 默认工具 ID
        "defaultPayloadId": 0, // 默认负载 ID
        "defaultUserCoorId": 1, // 默认用户坐标系 ID
        "type": "S20-180-ECO-V2", // 机器人型号
        "stateName": "未使能",
        "runDuration": 1374 // 运行时间
    }
}
```

15.5 机器人位姿主题

主题名称: publish/RobotPosture

推送数据:

```
{
    "ty": "publish/RobotPosture",
    "db": {
        "joint": [ 3.432,0.0,90.0,0.0,0.0,0.0 ], // 关节位置(单位: 弧度)
        "end": { // 关节位置(单位: 毫米, 弧度)
            "x": 203.002,
            "y": 1206.332,
            "z": 1066.0,
            "a": 0.0,
            "b": 90.0,
            "c": 93.432,
            "mode": -1
        },
        "ep": [] // 附加轴位置
    }
}
```

15.6 机器人坐标系主题

主题名称: publish/RobotCoordinate

推送数据:

```
{
    "ty": "publish/RobotCoordinate",
    "db": {
        "tool": { // 工具坐标系
            "x": 0.0,
            "y": 0.0,
            "z": 0.0,
            "a": 0.0,
            "b": 0.0,
            "c": 0.0
        },
```

```
        "user": { // 用户坐标系
            "x": 700.0000000000003,
            "y": -770.0,
            "z": 0.0,
            "a": 0.0,
            "b": 0.0,
            "c": 0.0
        }
    }
}
```

15.7 系统日志主题

主题名称: publish/Log

推送数据:

```
{
    "ty": "publish/Log",
    "db": [
        [6,13561,1760946003.582,"机器人发生<腕关节>奇异."],
        [6,13552,1760946003.582,"机器人构型获取失败."],
        [6,13552,1760946003.582,"机器人构型获取失败."],
        [6,13566,1760946003.582,"解析解逆解失败, 末端目标位姿: [0.8463706694236185,0.5380104024137184, 1.066,
0, 1.5707963267948966, 1.7468025738248967], 构型: -1."],
        [4,2023,1760946003.582,"奇异位置."]
}
```

15.7 系统错误主题

主题名称: publish/Error

推送数据:

```
{
    "ty": "publish/Log",
    "db": [
        [4,2023,1760946003.582,"奇异位置."]
}
```

16. 远程脚本模式接口

16.1 进入方式

1.通过 2.2 进入

2.运行脚本时，在主任务脚本中调用 enterRemoteScriptMode()指令

16.2 通信方式

使用 TCP/IP 协议，机器人控制器作为服务端，端口号为 9002。控制器默认 IP 地址为 192.168.1.136，若重新配置过 IP，则使用配置后的 IP 进行连接。

为方便数据解析，API 接口统一采用 Json 字符串进行数据通信，仅支持 UTF8 编码。

16.3 接口说明

通信端口: 9002

请求数据:

```
{
    "command": "resume", // 控制命令，当存在此字段时，其他字段无效// 可选值：resume: 恢复运行,stop：停止运行,interrupt：中断当前指令
    "script": "print(6)", // 要执行的脚本代码
    "vars": ["P1", "v1"], // 可选参数，执行完脚本后，将这些变量的值返回，如果工程被 stop 停止，则不返回
    "...": "..." // 自定义参数，例如"id": "mdo8zdy30wscc06e"
}
```

响应数据:

```
{
    "code": 0, // 0 表示成功，其他表示失败
    "msg": "OK", // 错误信息
    "vars": {
        "P1": 1, // 脚本执行后，返回的变量值
        "v2": "hello"
    },
    "...": "..." // 自定义参数会原值返回
}
```

16.4 运行机制

1. 通过上面的接口发送的脚本会被加入到运行队列中，按顺序执行，队列长度 64，队列满了之后会返回错误。
2. 每个请求中，脚本内定义的 local 类型的变量只在当前的指令脚本中生效。其他变量将持续生效。
3. 可用指令与正常编写脚本相同。
4. 如果执行类似 while(true)这种不会退出的脚本，可通过发送 interrupt 请求打断运行。
5. 因为远程脚本模式是在工程运行状态下执行的，可通过 2 中的相关工程运行控制指令来暂停、恢复或停止工程。

17.CRI 实时控制接口(Codroid RealTime Interface)

17.1 实时数据定义

```
// 机器人状态数据流
// 开始数据推送后，将向指定地址以二进制的形式，推送以下数据:
struct PushData {
    Bool isControlling{0};
    UInt8 errorCode{0};
    UInt8 nc[6]{0};
    Float64 jointPosition[6]{0};
    Float64 endPosition[6]{0};
}

// 机器人指令数据流
struct CommandData {
    Int64 timestamp{0}; // 目前版本未使用
    Float64 position[6]{0}; // 关节或末端位置
    UInt8 type{0}; // 0: 关节, 1: 末端
    UInt8 nc[7]{0}; // 保留字节
};
```

17.2 开启数据流推送

数据流通信协议：UDP

接口类型: CRI/StartDataPush

功能: 开启数据推送

请求数据:

```
{
    "id": "请求 ID",
    "ty": "CRI/StartDataPush",
    "db": {
      "ip": "192.168.1.200", // 填写推送目标地址
      "port": 9040, // 填写推送目标端口，合法范围为 1000-65534
      "duration": 1 // 数据推送间隔，单位: ms，范围: >=1 整数
    }
}
```

响应数据:

```
{
    "id":"请求 ID",
    "ty":"CRI/StartDataPush",
    "db":null
}
```

17.3 关闭数据流推送

数据流通信协议：UDP

接口类型: CRI/StopDataPush

功能: 开启数据推送

请求数据:

```
{
    "id": "请求 ID",
    "ty": "CRI/StopDataPush"
}
```

响应数据:

```
{
    "id":"1",
    "ty":"CRI/StopDataPush",
    "db":null
}
```

17.3 开启实时控制

数据流通信协议：UDP

接口类型: CRI/StartControl

功能: 开启数据推送

请求数据:

```
{
    "id": "请求 ID",
    "ty": "CRI/StartControl",
   "db": {
          "filterType": 0, // 0-关闭滤波 1-平均滤波值 2-二阶低通滤波 3-椭圆滤波
          "duration": 1, // 指令间隔，单位: ms，范围: [1, 16]整数
          "startBuffer": 3 // 启动缓冲点数量，范围: [1-100]整数，当接受到至少该数量的点位时，机器人才会开始运动
       }
}
```

响应数据:

```
{
    "id":"1",
    "ty":"CRI/StartControl",
    "db":null
}
```

17.3 关闭实时控制

数据流通信协议：UDP

接口类型: CRI/StopControl

功能: 开启数据推送

请求数据:

```
{
    "id": "请求 ID",
    "ty": "CRI/StopControl"
}
```

响应数据:

```
{
    "id":"1",
    "ty":"CRI/StopControl",
    "db":null
}
```

示例程序：示例程序和初版 PythonSDK 下载地址